
Chapter 8

Instance-Based Learning

lecture slides for textbook *Machine Learning*, ©Tom M. Mitchell, McGraw Hill, 1997

lecture slides of Raymond J. Mooney, University of Texas at Austin

Instance Based Learning

[Read Ch. 8]

- k -Nearest Neighbor
- Locally weighted regression
- Radial basis functions
- Case-based reasoning
- Lazy and eager learning

Instance-Based Learning

- Unlike other learning algorithms, does not involve construction of an explicit abstract generalization but classifies new instances based on direct comparison and similarity to known training instances.
- Training can be very easy, just memorizing training instances.
- Testing can be very expensive, requiring detailed comparison to all past training instances.
- Also known as:
 - Case-based
 - Exemplar-based
 - Nearest Neighbor
 - Memory-based
 - Lazy Learning

Similarity/Distance Metrics

- Instance-based methods assume a function for determining the similarity or distance between any two instances.
- For continuous feature vectors, Euclidian distance is the generic choice:

$$d(x_i, x_j) = \sqrt{\sum_{p=1}^n (a_p(x_i) - a_p(x_j))^2}$$

Where $a_p(x)$ is the value of the p th feature of instance x .

- For discrete features, assume distance between two values is 0 if they are the same and 1 if they are different (e.g. Hamming distance for bit vectors).
- To compensate for difference in units across features, scale all continuous values to the interval $[0,1]$.

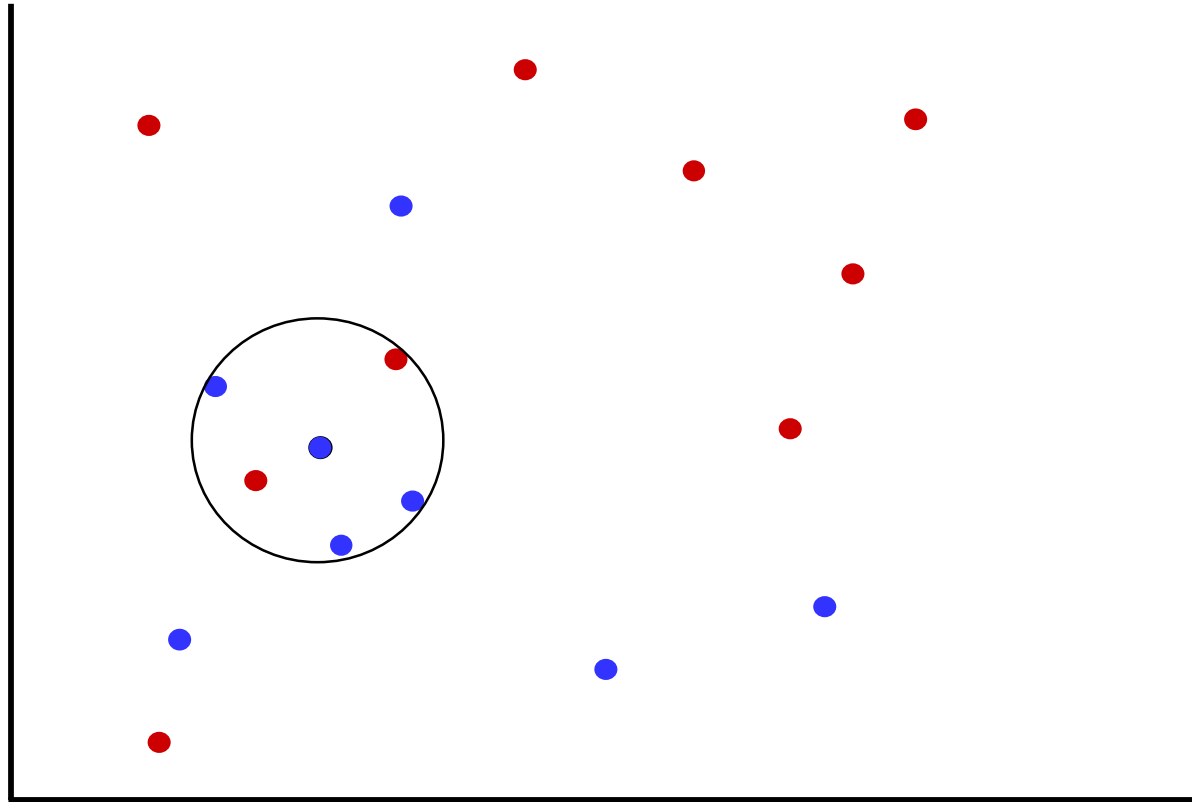
Other Distance Metrics

- Mahalanobis distance
 - Scale-invariant metric that normalizes for variance.
- Cosine Similarity
 - Cosine of the angle between the two vectors.
 - Used in text and other high-dimensional data.
- Pearson correlation
 - Standard statistical correlation coefficient.
 - Used for bioinformatics data.
- Edit distance
 - Used to measure distance between unbounded length strings.
 - Used in text and bioinformatics.

K-Nearest Neighbor

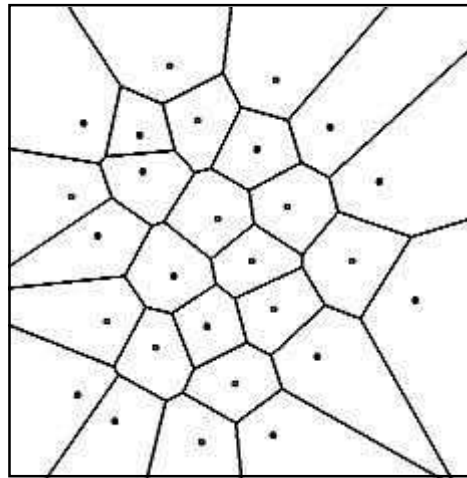
- Calculate the distance between a test point and every training instance.
- Pick the k closest training examples and assign the test instance to the most common category amongst these nearest neighbors.
- Voting multiple neighbors helps decrease susceptibility to noise.
- Usually use odd value for k to avoid ties.

5-Nearest Neighbor Example



Implicit Classification Function

- Although it is not necessary to explicitly calculate it, the learned classification rule is based on regions of the feature space closest to each training example.
- For 1-nearest neighbor with Euclidian distance, the **Voronoi diagram** gives the complex polyhedra segmenting the space into the regions closest to each point.



Behavior in the Limit

Consider $p(x)$ defines probability that instance x will be labeled 1 (positive) versus 0 (negative).

Nearest neighbor:

- As number of training examples $\rightarrow \infty$, approaches Gibbs Algorithm

Gibbs: with probability $p(x)$ predict 1, else 0

k -Nearest neighbor:

- As number of training examples $\rightarrow \infty$ and k gets large, approaches Bayes optimal

Bayes optimal: if $p(x) > .5$ then predict 1, else 0

Note Gibbs has at most twice the expected error of Bayes optimal

Efficient Indexing

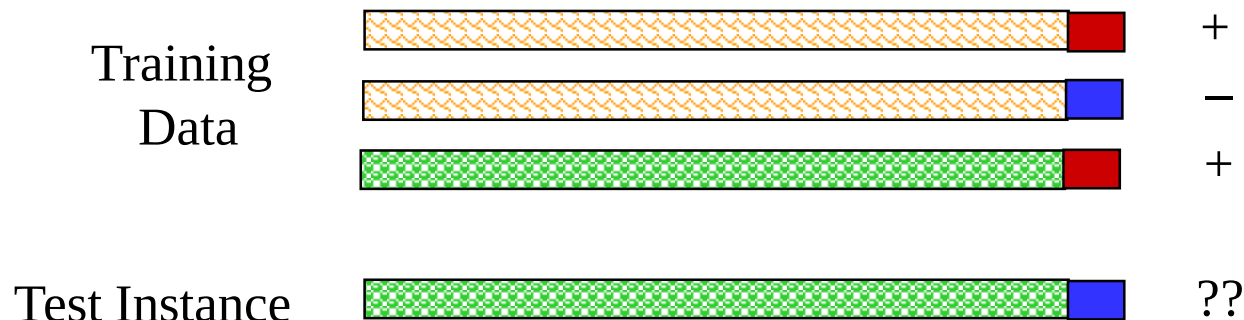
- Linear search to find the nearest neighbors is not efficient for large training sets.
- Indexing structures can be built to speed testing.
- For Euclidian distance, a **kd-tree** can be built that reduces the expected time to find the nearest neighbor to $O(\log n)$ in the number of training examples.
 - Nodes branch on threshold tests on individual features and leaves terminate at nearest neighbors.
- Other indexing structures possible for other metrics or string data.
 - Inverted index for text retrieval.

Nearest Neighbor Variations

- Can be used to estimate the value of a real-valued function (regression) by taking the average function value of the k nearest neighbors to an input point.
- All training examples can be used to help classify a test instance by giving every training example a vote that is weighted by the inverse square of its distance from the test instance.

Feature Relevance and Weighting

- Standard distance metrics weight each feature equally when determining similarity.
 - Problematic if many features are irrelevant, since similarity along many irrelevant examples could mislead the classification.
- Features can be weighted by some measure that indicates their ability to discriminate the category of an example, such as information gain.
- Overall, instance-based methods favor global similarity over concept simplicity.



Distance-Weighted k NN

Might want weight nearer neighbors more heavily...

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^k w_i f(x_i)}{\sum_{i=1}^k w_i}$$

where

$$w_i \equiv \frac{1}{d(x_q, x_i)^2}$$

and $d(x_q, x_i)$ is distance between x_q and x_i

Note now it makes sense to use *all* training examples instead of just k

→ Shepard's method

Curse of Dimensionality

Imagine instances described by 20 attributes, but only 2 are relevant to target function

Curse of dimensionality: nearest nbr is easily mislead when high-dimensional X

One approach:

- Stretch j th axis by weight z_j , where $z_1, \dots, z_n$ chosen to minimize prediction error
- Use cross-validation to automatically choose weights $z_1, \dots, z_n$
- Note setting z_j to zero eliminates this dimension altogether

see [Moore and Lee, 1994]

Locally Weighted Regression

Note k NN forms local approximation to f for each query point x_q

Why not form an explicit approximation $\hat{f}(x)$ for region surrounding x_q

- Fit linear function to k nearest neighbors
- Fit quadratic, ...
- Produces “piecewise approximation” to f

Several choices of error to minimize:

- Squared error over k nearest neighbors

$$E_1(x_q) \equiv \frac{1}{2} \sum_{x \in k \text{ nearest nbrs of } x_q} (f(x) - \hat{f}(x))^2$$

- Distance-weighted squared error over all nbrs

$$E_2(x_q) \equiv \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2 K(d(x_q, x))$$

- ...

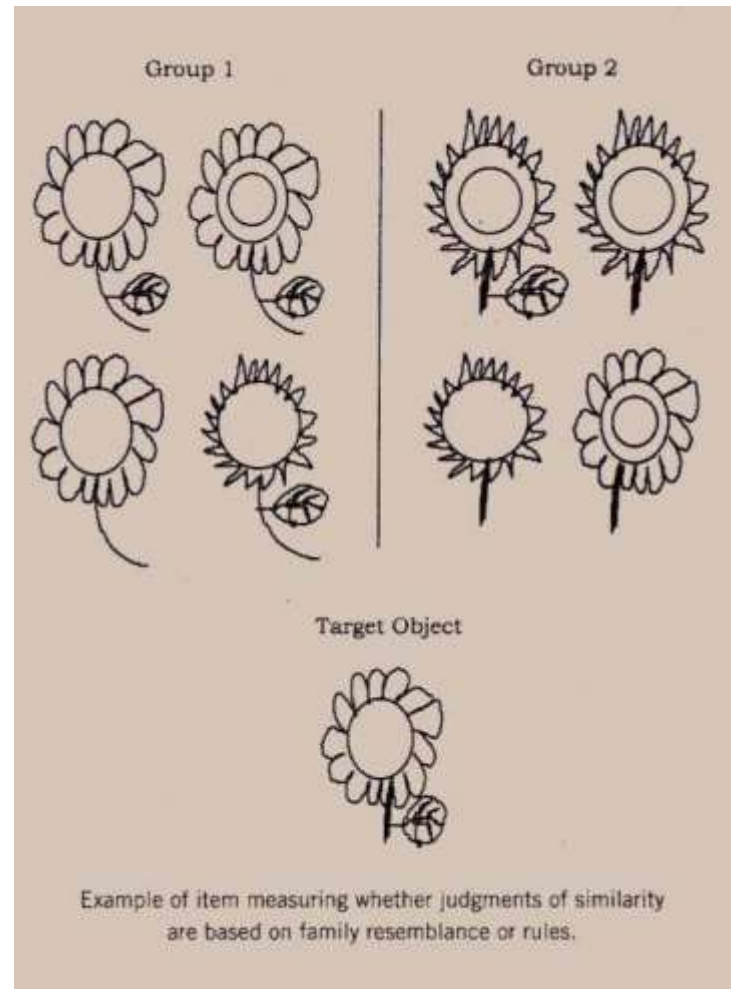
Approximate the target function f near x_q using a linear function

$$f(x) = w_0 + w_1 a_1(x) + w_2 a_2(x) + \dots + w_n a_n(x)$$

Where $a_i(x)$ denotes the i^{th} attribute of the instance x .

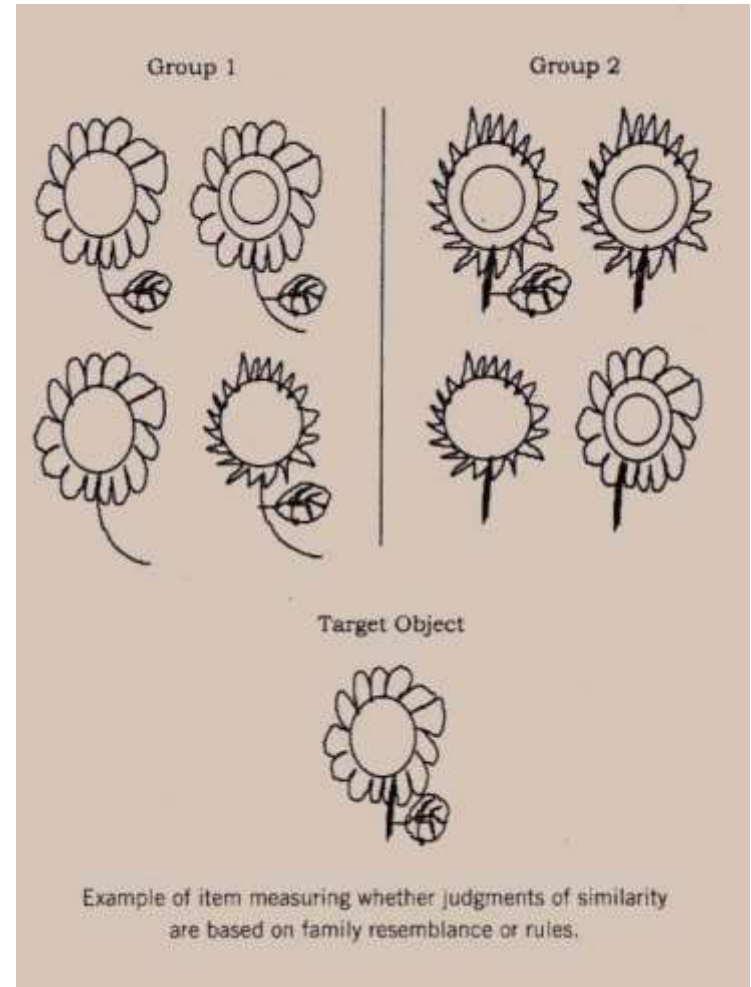
Use methods such as Gradient descent to find the coefficients $w_0, w_1, \dots, w_n$ to minimize the error in fitting such linear functions to a given set of training examples.

How do you classify the target object ?



Rules and Instances in Human Learning Biases

- Psychological experiments show that people from different cultures exhibit distinct categorization biases.
- “Western” subjects favor simple rules (straight stem) and classify the target object in group 2.
- “Asian” subjects favor global similarity and classify the target object in group 1.



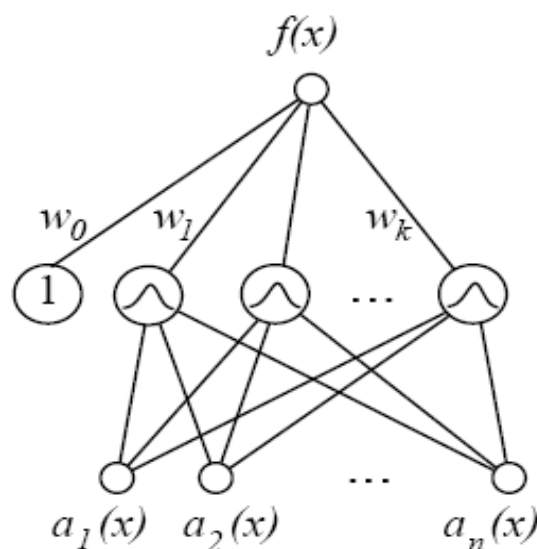
Other Issues

- Can reduce storage of training instances to a small set of representative examples.
 - Support vectors in an SVM are somewhat analogous.
- Can hybridize with rule-based methods or neural-net methods.
 - Radial basis functions in neural nets and Gaussian kernels in SVMs are similar.
- Can be used for more complex relational or graph data.
 - Similarity computation is complex since it involves some sort of graph isomorphism.
- Can be used in problems other than classification.
 - Case-based planning
 - Case-based reasoning in law and business.

Radial Basis Function Networks

- Global approximation to target function, in terms of linear combination of local approximations
- Used, e.g., for image classification
- A different kind of neural network
- Closely related to distance-weighted regression, but “eager” instead of “lazy”

Radial Basis Function Networks



where $a_i(x)$ are the attributes describing instance x , and

$$f(x) = w_0 + \sum_{u=1}^k w_u K_u(d(x_u, x))$$

One common choice for $K_u(d(x_u, x))$ is

$$K_u(d(x_u, x)) = e^{-\frac{1}{2\sigma_u^2}d^2(x_u, x)}$$

Training Radial Basis Function Networks

Q1: What x_u to use for each kernel function

$K_u(d(x_u, x))$

- Scatter uniformly throughout instance space
- Or use training instances (reflects instance distribution)

Q2: How to train weights (assume here Gaussian

K_u)

- First choose variance (and perhaps mean) for each K_u
 - e.g., use EM
- Then hold K_u fixed, and train linear output layer
 - efficient methods to fit linear function

Case-Based Reasoning

Can apply instance-based learning even when
 $X \neq \mathbb{R}^n$

→ need different “distance” metric

Case-Based Reasoning is instance-based learning
applied to instances with symbolic logic
descriptions

```
((user-complaint error53-on-shutdown)
 (cpu-model PowerPC)
 (operating-system Windows)
 (network-connection PCIA)
 (memory 48meg)
 (installed-applications Excel Netscape VirusScan)
 (disk 1gig)
 (likely-cause ???))
```

Case-Based Reasoning in CADET

CADET: 75 stored examples of mechanical devices

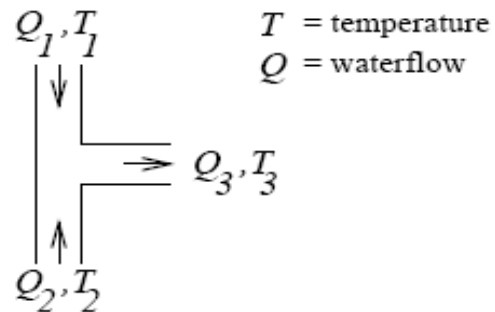
- each training example: $\langle$ qualitative function, mechanical structure $\rangle$
- new query: desired function,
- target value: mechanical structure for this function

Distance metric: match qualitative function descriptions

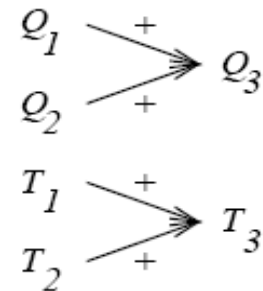
Case-Based Reasoning in CADET

A stored case: T-junction pipe

Structure:



Function:

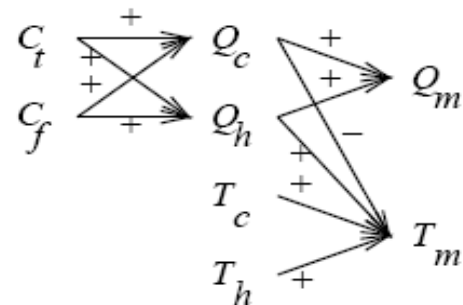


A problem specification: Water faucet

Structure:

?

Function:



Case-Based Reasoning in CADET

- Instances represented by rich structural descriptions
- Multiple cases retrieved (and combined) to form solution to new problem
- Tight coupling between case retrieval and problem solving

Bottom line:

- Simple matching of cases useful for tasks such as answering help-desk queries
- Area of ongoing research

Lazy and Eager Learning

Lazy: wait for query before generalizing

- k -NEAREST NEIGHBOR, Case based reasoning

Eager: generalize before seeing query

- Radial basis function networks, ID3, Backpropagation, NaiveBayes, ...

Does it matter?

- Eager learner must create global approximation
- Lazy learner can create many local approximations
- if they use same H , lazy can represent more complex fns (e.g., consider H = linear functions)

Conclusions

- IBL methods classify test instances based on similarity to specific training instances rather than forming explicit generalizations.
- Typically trade decreased training time for increased testing time.